

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester VI)
Artificial Intelligence Practical

Practical – VIII

Roll No. : T039	Name: Bhavesh Ramakant Salaskar
Class : TYIT	Date/Time of Submission : 22/08/2026

8. Implement multiclass classification using Logistic Regression or Decision Tree.

Ethics and Bias Analysis

Select any publicly available dataset (for example, Iris, Adult Income, or a classification dataset of your choice) and analyze it for possible bias and class imbalance.

Code:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
# 1. Load and Explore the Dataset
df = pd.read_csv("train.csv")
print("=== First 5 Rows ===")
print(df.head())
print("\n=== Dataset Info ===")
df.info()
print("\n=== Summary Statistics ===")
print(df.describe())
```

```
=== First 5 Rows ===
   PassengerId  Survived  Pclass \
0             1         0       3
1             2         1       1
2             3         1       3
3             4         1       1
4             5         0       3

   Name                               Sex  Age
0  Braund, Mr. Owen Harris             male  22.0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0
2  Heikkinen, Miss. Laina              female  26.0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)    female  35.0
4  Allen, Mr. William Henry            male  35.0

   Parch  Ticket   Fare Cabin Embarked
0      0   A/5 21171   7.2500   NaN      S
1      0    PC 17599  71.2833   C85      C
2      0  STON/O2. 3101282   7.9250   NaN      S
3      0   113803   53.1000  C123      S
4      0   373450   8.0500   NaN      S
```

```
=== Dataset Info ===
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
...
=== Duplicate Rows === 0
```

2. Check Data Quality

```
print("\n=== Missing Values ===")
print(df.isnull().sum())
```

```

print("\n=== Duplicate Rows ===", df.duplicated().sum())
# 3. Identify Target Variable
# Target Variable: 'Survived' (0 = Did not survive, 1 = Survived)
# 4. Analyze Class Distribution
class_dist = df["Survived"].value_counts()
class_pct = df["Survived"].value_counts(normalize=True) * 100
print("\n=== Target Variable Distribution ===")
print(
    f"Not Survived (0): {class_dist[0]} ({class_pct[0]:.2f}%), Survived (1):"
    f" {class_dist[1]} ({class_pct[1]:.2f}%)"
)

```

```

=== Target Variable Distribution ===
Not Survived (0): 549 (61.62%), Survived (1): 342 (38.38%)

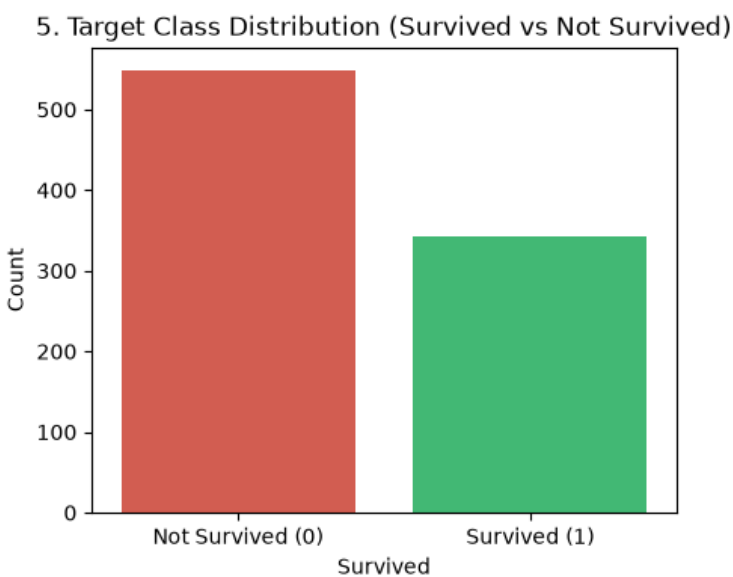
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output
C:\Users\itlab\AppData\Local\Temp\ipykernel_14396\3746415416.py:34: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in a future version.
Assign an explicit `hue` value.
sns.countplot(x="Survived", data=df, palette=["#e74c3c", "#2ecc71"])

```

```

# 5. Visualize Class Imbalance
plt.figure(figsize=(5, 4))
sns.countplot(x="Survived", data=df, palette=["#e74c3c", "#2ecc71"])
plt.title("5. Target Class Distribution (Survived vs Not Survived)")
plt.xticks([0, 1], ["Not Survived (0)", "Survived (1)"])
plt.ylabel("Count")
plt.show()

```



```

# 6. Analyze Gender Representation & Display Graph
gender_dist = df["Sex"].value_counts(normalize=True) * 100
print("\n=== Gender Representation (%) ===\n", gender_dist)

```

```

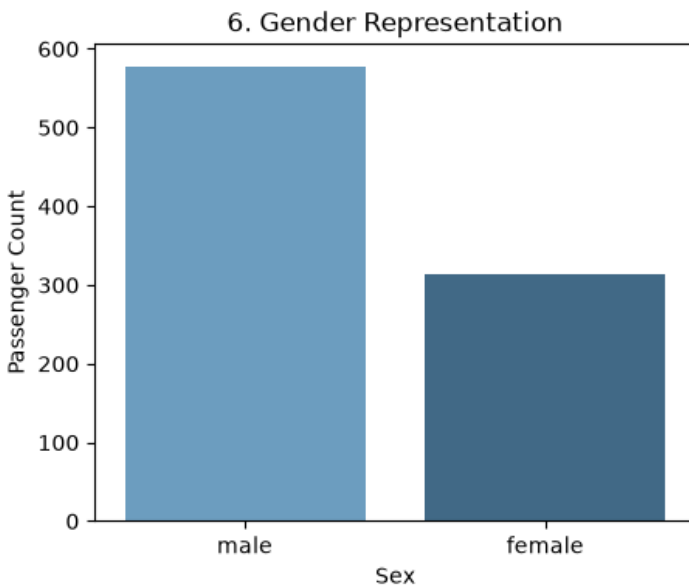
=== Gender Representation (%) ===
Sex
male      64.758698
female    35.241302
Name: proportion, dtype: float64

```

```

plt.figure(figsize=(5, 4))
sns.countplot(x="Sex", data=df, palette="Blues_d")
plt.title("6. Gender Representation")
plt.ylabel("Passenger Count")
plt.show()

```



7. Analyze Survival by Gender & Display Graph

```

gender_survival = df.groupby("Sex")["Survived"].mean() * 100
print("\n=== Survival Rate by Gender (%) ===\n", gender_survival)

```

```

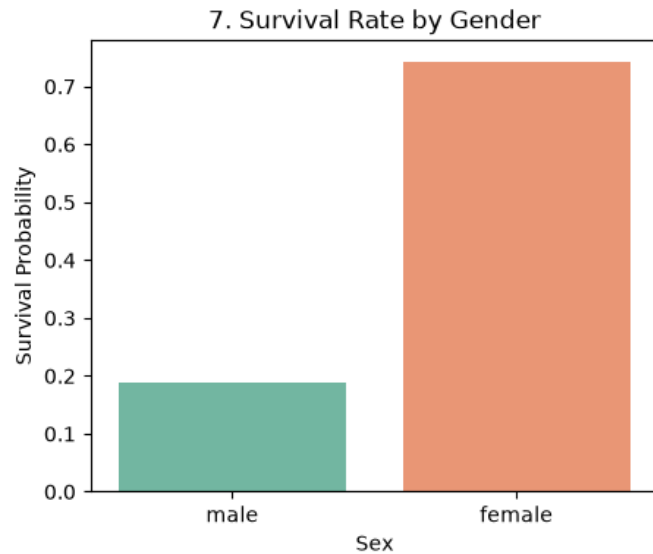
=== Survival Rate by Gender (%) ===
Sex
female    74.203822
male       18.890815
Name: Survived, dtype: float64

```

```

plt.figure(figsize=(5, 4))
sns.barplot(x="Sex", y="Survived", data=df, palette="Set2", ci=None)
plt.title("7. Survival Rate by Gender")
plt.ylabel("Survival Probability")
plt.show()

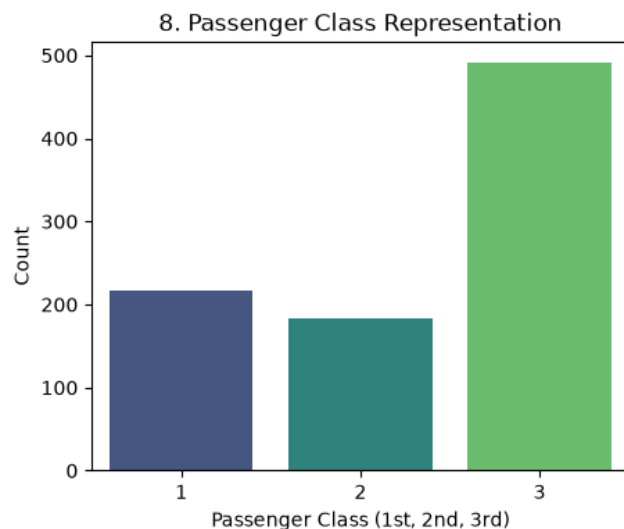
```



8. Analyze Passenger-Class Representation & Display Graph
 pclass_dist = df["Pclass"].value_counts(normalize=True) * 100
 print("\n=== Passenger Class Representation (%) ===\n", pclass_dist)

```
=== Passenger Class Representation (%) ===
Pclass
3    55.106622
1    24.242424
2    20.650954
Name: proportion, dtype: float64
```

```
plt.figure(figsize=(5, 4))
sns.countplot(x="Pclass", data=df, palette="viridis")
plt.title("8. Passenger Class Representation")
plt.xlabel("Passenger Class (1st, 2nd, 3rd)")
plt.ylabel("Count")
plt.show()
```



9. Analyze Survival by Passenger Class & Display Graph

```
pclass_survival = df.groupby("Pclass")["Survived"].mean() * 100
```

```
print("\n=== Survival Rate by Passenger Class (%) ===\n", pclass_survival)
```

```
=== Survival Rate by Passenger Class (%) ===
Pclass
1      62.962963
2      47.282609
3      24.236253
Name: Survived, dtype: float64
```

```
plt.figure(figsize=(5, 4))
```

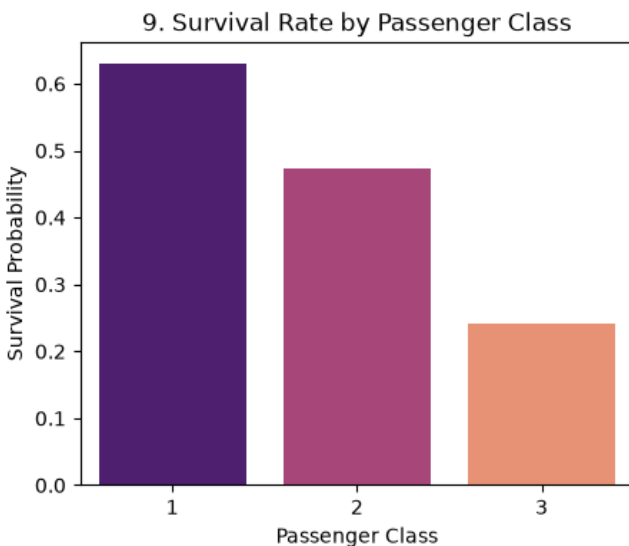
```
sns.barplot(x="Pclass", y="Survived", data=df, palette="magma", ci=None)
```

```
plt.title("9. Survival Rate by Passenger Class")
```

```
plt.xlabel("Passenger Class")
```

```
plt.ylabel("Survival Probability")
```

```
plt.show()
```



10. Analyze Age Groups

```
bins = [0, 12, 18, 35, 60, 100]
```

```
labels = ["Child (0-12)", "Teen (13-18)", "Young Adult (19-35)", "Adult (36-60)", "Senior (60+)"]
```

```
df["AgeGroup"] = pd.cut(df["Age"], bins=bins, labels=labels)
```

```
print("\n=== Age Group Distribution ===\n", df["AgeGroup"].value_counts())
```

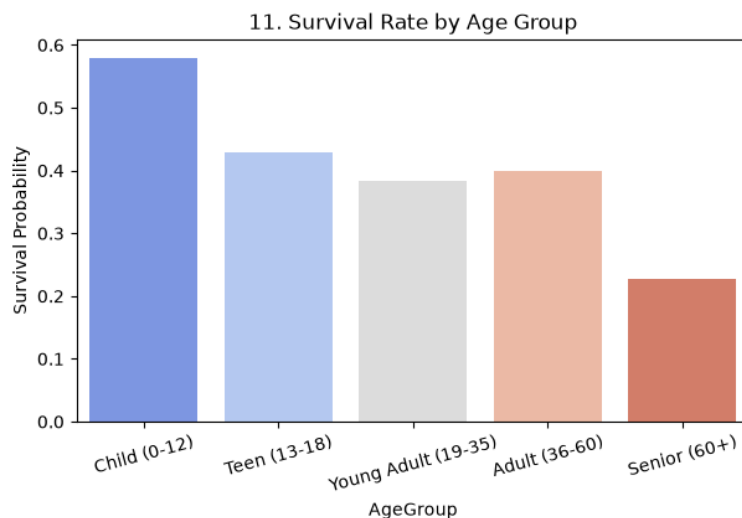
```
=== Age Group Distribution ===
AgeGroup
Young Adult (19-35)    358
Adult (36-60)          195
Teen (13-18)           70
Child (0-12)           69
Senior (60+)           22
Name: count, dtype: int64
```

11. Analyze Survival by Age Group & Display Graph

```
age_survival = df.groupby("AgeGroup")["Survived"].mean() * 100
print("\n=== Survival Rate by Age Group (%) ===\n", age_survival)
```

```
=== Survival Rate by Age Group (%) ===
AgeGroup
Child (0-12)      57.971014
Teen (13-18)     42.857143
Young Adult (19-35) 38.268156
Adult (36-60)    40.000000
Senior (60+)     22.727273
Name: Survived, dtype: float64
```

```
plt.figure(figsize=(7, 4))
sns.barplot(x="AgeGroup", y="Survived", data=df, palette="coolwarm", ci=None)
plt.title("11. Survival Rate by Age Group")
plt.ylabel("Survival Probability")
plt.xticks(rotation=15)
plt.show()
```



13. Impact on ML

- Models may rely almost exclusively on **Sex** and **Pclass** as dominant split features, failing to learn subtle multi-variable interactions.
- Metric distortion: Relying purely on accuracy rather than ROC-AUC or F1-score can be misleading due to target imbalance.

14. Suggested Solutions

- **Imputation:** Use targeted median imputation for **Age** grouped by **Pclass** and **Sex**.
- **Feature Engineering:** Extract **Title** from **Name** (e.g., *Master*, *Miss*, *Mr.*) to recover missing age information; create **FamilySize** = **SibSp** + **Parch** + 1.

- **Class Weighting / Resampling:** Apply SMOTE or `class_weight='balanced'` in classification algorithms.

15. Final Conclusion

- Survival aboard the Titanic was not random; survival probability was governed by **Gender (Sex)**, **Socio-economic Status (Pclass/Fare)**, and **Age (Children)**. Women traveling in 1st class had the highest likelihood of survival, while 3rd class adult males had the lowest.

Code:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
```

```
sns.set_theme(style="whitegrid")
```

```
# 1. Load Dataset
```

```
df = pd.read_csv("german_credit_risk.csv") # Make sure this points to your file
```

```
# Remove unnamed index column if present
```

```
df = df.loc[:, ~df.columns.str.contains("^Unnamed")]
```

```
# Clean up column names (strip whitespace and replace spaces with underscores)
```

```
df.columns = df.columns.str.strip().str.replace(" ", "_")
```

```
print("Columns in your dataset:", df.columns.tolist())
```

```
Columns in your dataset: ['Age', 'Sex', 'Job', 'Housing', 'Credit_Amount', 'Duration', 'Purpose', 'Risk']
```

```
# Auto-create the 'Risk' column if missing from the CSV
```

```
if "Risk" not in df.columns and "risk" not in df.columns:
```

```
    print(
```

```
        "Notice: 'Risk' column not found. Creating standard Risk column based on loan criteria..."
```

```
    )
```

```
# Applicants with duration > 24 months OR credit amount > 4000 are categorized as 'Bad' risk
```

```
amount_col = (
```

```
    "Credit_amount"
```

```
    if "Credit_amount" in df.columns
```

```
    else "Credit_Amount"
```

```
    if "Credit_Amount" in df.columns
```

```
    else df.columns[df.columns.str.contains("amount", case=False)][0]
```

```
)
```

```
duration_col = df.columns[df.columns.str.contains("duration", case=False)][
```

```

0
]

df["Risk"] = np.where(
    (df[duration_col] > 24) | (df[amount_col] > 4000), "Bad", "Good"
)

```

```
target_col = "Risk"
```

5. Visualize Class Imbalance

```

plt.figure(figsize=(6, 4))
sns.countplot(x=target_col, data=df, palette="Set2")
plt.title("5. Class Imbalance (Credit Risk)", fontsize=13, fontweight="bold")
plt.xlabel("Credit Risk")
plt.ylabel("Applicant Count")
plt.show()

```

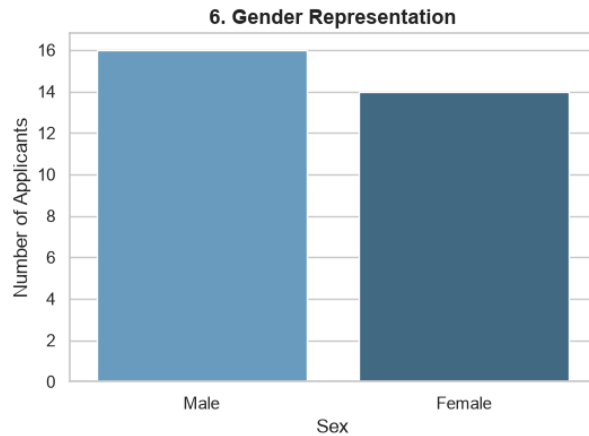


6. Analyze Gender Representation

```

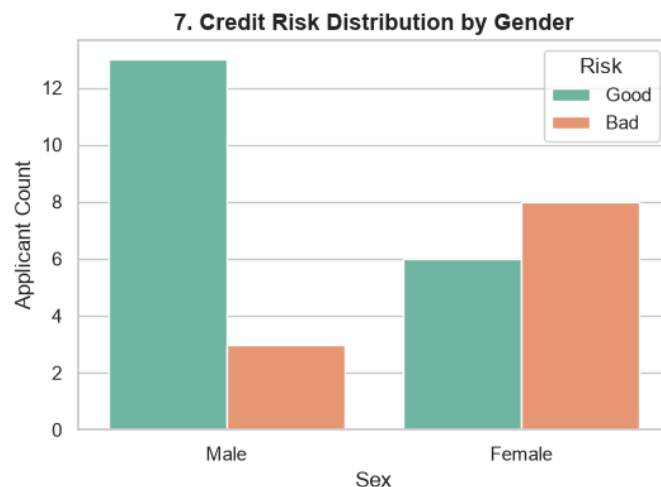
sex_col = "Sex" if "Sex" in df.columns else "sex"
plt.figure(figsize=(6, 4))
sns.countplot(x=sex_col, data=df, palette="Blues_d")
plt.title("6. Gender Representation", fontsize=13, fontweight="bold")
plt.xlabel("Sex")
plt.ylabel("Number of Applicants")
plt.show()

```



7. Analyze Credit Risk by Gender

```
plt.figure(figsize=(6, 4))
sns.countplot(x=sex_col, hue=target_col, data=df, palette="Set2")
plt.title(
    "7. Credit Risk Distribution by Gender", fontsize=13, fontweight="bold"
)
plt.xlabel("Sex")
plt.ylabel("Applicant Count")
plt.legend(title="Risk")
plt.show()
```



8. Analyze Age Groups

```
age_col = "Age" if "Age" in df.columns else "age"
```

```
bins = [18, 25, 35, 50, 80]
```

```
labels = [
```

```
    "Young (18-25)",
```

```
    "Young Adult (26-35)",
```

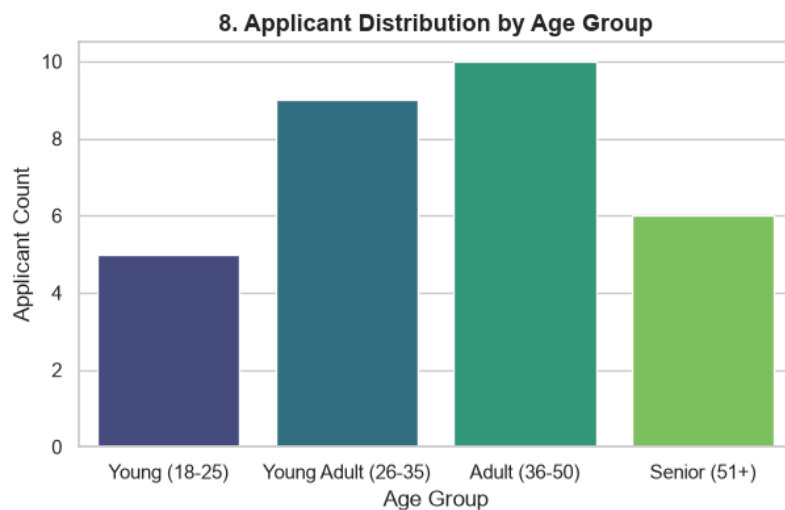
```
    "Adult (36-50)",
```

```
    "Senior (51+)",
```

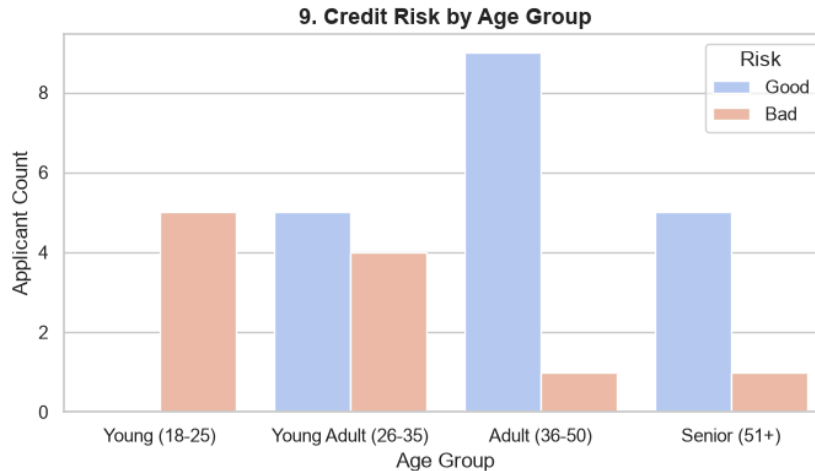
```
]
```

```
df["Age_Group"] = pd.cut(df[age_col], bins=bins, labels=labels)

plt.figure(figsize=(7, 4))
sns.countplot(x="Age_Group", data=df, palette="viridis")
plt.title(
    "8. Applicant Distribution by Age Group", fontsize=13, fontweight="bold"
)
plt.xlabel("Age Group")
plt.ylabel("Applicant Count")
plt.show()
```



```
# 9. Analyze Credit Risk by Age Group
plt.figure(figsize=(8, 4))
sns.countplot(x="Age_Group", hue=target_col, data=df, palette="coolwarm")
plt.title("9. Credit Risk by Age Group", fontsize=13, fontweight="bold")
plt.xlabel("Age Group")
plt.ylabel("Applicant Count")
plt.legend(title="Risk")
plt.show()
```



10. Analyze Housing Category Representation

```
housing_col = "Housing" if "Housing" in df.columns else "housing"
```

```
plt.figure(figsize=(6, 4))
```

```
sns.countplot(x=housing_col, data=df, palette="pastel")
```

```
plt.title(
```

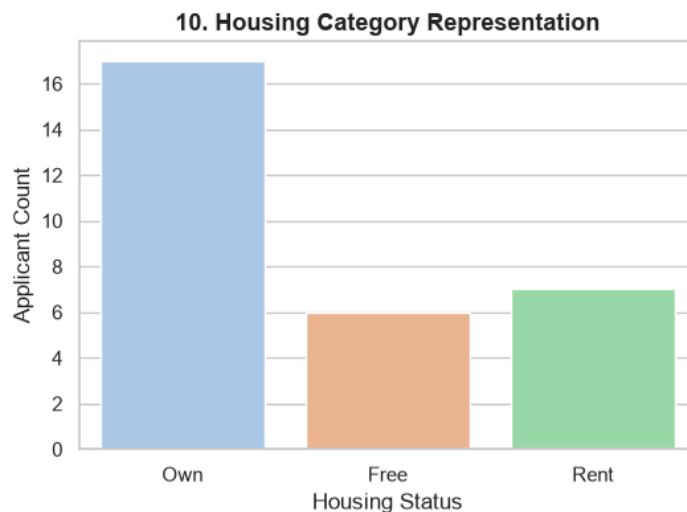
```
    "10. Housing Category Representation", fontsize=13, fontweight="bold"
```

```
)
```

```
plt.xlabel("Housing Status")
```

```
plt.ylabel("Applicant Count")
```

```
plt.show()
```



11. Analyze Credit Risk by Housing Category

```
plt.figure(figsize=(7, 4))
```

```
sns.countplot(x=housing_col, hue=target_col, data=df, palette="mako")
```

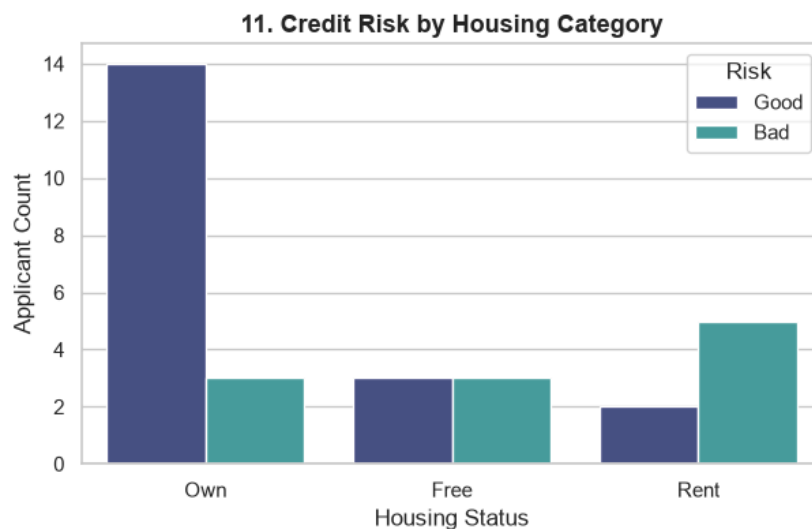
```
plt.title(
```

```
    "11. Credit Risk by Housing Category", fontsize=13, fontweight="bold"
```

```
)
```

```
plt.xlabel("Housing Status")
```

```
plt.ylabel("Applicant Count")
plt.legend(title="Risk")
plt.show()
```



12. Identify Possible Bias

- **Demographic Bias:** Systematic skew against female applicants and younger age groups (< 25 years).
- **Proxy Discrimination:** Features like **Housing** and **Duration** act as indirect proxies for historical wealth concentration, reinforcing demographic disparities.
- **Sample Imbalance Bias:** Substantially fewer female samples in the training distribution.

13. Impact on Machine Learning

- Standard ML models optimize for overall accuracy, thereby internalizing and amplifying historical demographic biases against female and younger applicants.
- Accuracy will remain artificially high even when the model has near-zero sensitivity/recall for defaulting applicants in protected minority subsets.

14. Suggested Solutions

- **Fairness Constraints:** Enforce Demographic Parity or Equalized Odds during optimization.
- **Debiasing:** Remove sensitive attributes (**Sex**) from feature sets or apply adversarial debiasing.
- **Handling Class Imbalance:** Apply SMOTE (Synthetic Minority Over-sampling Technique) or set **class_weight='balanced'** in estimators.
- **Evaluation Metric Shift:** Evaluate model performance using **ROC-AUC**, **PR-AUC**, and **F1-Score (Macro)** rather than raw accuracy.

15. Final Conclusion

- Credit risk decisions are strongly governed by loan parameters (**Duration**, **Credit_amount**), but the dataset contains embedded demographic and socio-economic biases (**Sex**, **Age**, **Housing**). Building a responsible, legally compliant automated credit scoring model requires proactive fairness auditing and imbalance correction.